

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Объектно-ориентированное программирование»

Студент гр. 4384

Серженко Д.К.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2025

ЦЕЛЬ РАБОТЫ

Разработать архитектуру пошаговой стратегической игры с применением объектно-ориентированного подхода. Создать систему взаимодействующих игровых объектов (игрок, враги, здания) на клеточном поле с препятствиями и специальными клетками. Реализовать базовые игровые механики: перемещение, систему боя с двумя режимами атаки, спавн врагов. Освоить работу с динамической памятью через реализацию конструкторов копирования и перемещения для игрового поля. Закрепить принципы инкапсуляции и безопасного управления ресурсами в C++.

ЗАДАНИЕ

1. Создать интерфейс карточки заклинания. Заклинание должно применяться игроком. На использование заклинания игрок тратит один ход.
2. Создать класс “руки” игрока, которая содержит все карточки заклинаний, которые игрок может применить в свой ход. Изначально рука игрока содержит только одно случайное заклинание. Реализовать возможность получать новые заклинание игроком, например, тратить очки на покупку или после уничтожения определенного кол-ва врагов. Размер “руки” должен быть ограничен и задается через конструктор.
3. Реализовать интерфейс заклинанием прямого урона. Это заклинание при использовании должно наносить урон врагу или вражескому зданию, если они находятся в достижимом радиусе. Если в качестве цели не выбран враг или вражеское здание, то заклинание не используется.
4. Реализовать интерфейс заклинания урона по площади. Это заклинание при использовании в допустимом радиусе наносит урон по области 2 на 2 клетки. Заклинание используется, даже если там нет никого.

Примечания:

- Интерфейс заклинания должен быть унифицирован, чтобы их можно было единообразно использовать через интерфейс. Не должно быть методов в интерфейсе, которые не используются каким-то классом наследником.
- Избегайте явных проверок на тип данных.

ВЫПОЛНЕНИЕ РАБОТЫ

1. Общая архитектура системы

Архитектура игры расширена добавлением заклинаний, механики "руки".

Основные компоненты:

- ISpell – абстрактный интерфейс для всех заклинаний
- ITarget - интерфейс для объектов, которые могут быть целью заклинаний
- DirectDamageSpell - заклинание прямого урона
- AreaDamageSpell - заклинание урона по площади
- Hand - класс для управления заклинаниями
- Обновленный Player с поддержкой заклинаний
- Обновленный Game с механиками выбора и использования заклинаний

2. Иерархия классов и их назначение

2.1. Интерфейс ITarget

Назначение: интерфейс для всех объектов, которые могут быть целью заклинаний.

```
class ITarget {
public:
    virtual ~ITarget() = default;
    virtual void takeDamage(int damage) = 0;
    virtual bool isAlive() const = 0;
    virtual Point getPosition() const = 0;
    virtual int getHealth() const = 0;
};
```

Обоснование: использование интерфейса позволяет создавать заклинания, которые могут работать с любыми объектами, реализующими этот интерфейс.

2.2. Интерфейс ISpell

Назначение: абстрактный класс для всех заклинаний.

```
class ISpell {
public:
    virtual ~ISpell() = default;
    virtual std::string getName() const = 0;
    virtual int getManaCost() const = 0;
    virtual int getCastRange() const = 0;
    virtual bool canCast(const Point& casterPosition,
                        const Point& targetPosition) const = 0;
    virtual void cast(const Point& casterPosition,
                    const Point& targetPosition) = 0;
    virtual std::unique_ptr<ISpell> clone() const = 0;
};
```

Обоснование: Позволяет создавать различные заклинания с единым интерфейсом использования. Метод clone() создаёт копии заклинаний.

2.3. Класс DirectDamageSpell

Назначение: заклинание прямого урона по одной цели.

```
class DirectDamageSpell : public ISpell {
public:
    DirectDamageSpell(std::string name, int damage,
                    int manaCost, int castRange);
    std::string getName() const override;
    int getManaCost() const override;
    int getCastRange() const override;
    bool canCast(const Point& casterPosition,
                const Point& targetPosition) const override;
    void cast(const Point& casterPosition,
            const Point& targetPosition) override;
    std::unique_ptr<ISpell> clone() const override;
    static void setAvailableTargets(
        const std::vector<std::shared_ptr<ITarget>>& targets);

private:
    std::string name;
    int damage;
    int manaCost;
    int castRange;

    static std::vector<std::shared_ptr<ITarget>> availableTargets;
    std::shared_ptr<ITarget> findTargetAtPosition(
        const Point& position) const;
};
```

Обоснование: отдельный класс для заклинаний прямого урона позволяет добавлять новые эффекты. Использование статического поля для доступных целей обеспечивает единую точку обновления для всех заклинаний.

2.4. Класс AreaDamageSpell

Назначение: заклинание урона по площади.

```
class AreaDamageSpell : public ISpell {
public:
    AreaDamageSpell(std::string name, int damage,
                    int manaCost, int castRange, int areaSize = 2);

    std::string getName() const override;
    int getManaCost() const override;
    int getCastRange() const override;
    bool canCast(const Point& casterPosition,
                 const Point& targetPosition) const override;
    void cast(const Point& casterPosition,
              const Point& targetPosition) override;
    std::unique_ptr<ISpell> clone() const override;

    static void setAvailableTargets(
        const std::vector<std::shared_ptr<ITarget>>& targets);

private:
    std::string name;
    int damage;
    int manaCost;
    int castRange;
    int areaSize;

    static std::vector<std::shared_ptr<ITarget>> availableTargets;

    std::vector<std::shared_ptr<ITarget>> findTargetsInArea(
        const Point& center) const;
    bool isInArea(const Point& point, const Point& center) const;
};
```

Обоснование: отдельная реализация для заклинания по площади показывает преимущества полиморфизма, один интерфейс, разные реализации.

2.5. Класс Hand

Назначение: управление заклинаниями, доступными игроку.

```
class Hand {
public:
    explicit Hand(int capacity = 2);
```

```

    bool addSpell(std::unique_ptr<ISpell> spell);
    bool removeSpell(int index);
    const ISpell* getSpell(int index) const;
    ISpell* getSpell(int index);
    bool useSpell(int index, const Point& casterPosition,
                  const Point& targetPosition);

    bool isFull() const;
    bool isEmpty() const;
    int getSize() const;
    int getCapacity() const;

    void drawRandomSpell(
        const std::vector<std::unique_ptr<ISpell>>& spellDeck);

    void display() const;

private:
    std::vector<std::unique_ptr<ISpell>> spells;
    int capacity;

    bool checkInvariant() const;
};

```

Обоснование: класс инкапсулирует логику управления заклинаниями, обеспечивает проверку инвариантов и предотвращает ошибки использования. Использование `std::unique_ptr` гарантирует безопасное управление памятью.

2.6. Обновленный класс Player

Назначение: расширение класса игрока для поддержки заклинаний.

```

class Player {
public:
    Player(int health = 100, int damage = 10,
           int mana = 30, Point position = Point(0, 0));

    int getMana() const;
    void setMana(int newMana);
    void restoreMana(int amount);
    Hand& getHand();
    const Hand& getHand() const;

private:
    int mana;
    int maxMana;
    Hand hand;
};

```

Обоснование: использование композиции для добавления системы заклинаний.

2.7. Обновленный класс Game

Назначение: расширение для управления заклинаниями.

```
class Game {
public:
    Game(int width, int height, int enemyCount);
    void run();

private:

    std::vector<std::unique_ptr<ISpell>> spellDeck;
    std::vector<std::shared_ptr<ITarget>> allTargets;

    void initializeSpellDeck();
    void updateTargetsList();
    bool processSpellCast();
    void checkForSpellRewards();
};
```

3. Система заклинаний

3.1. Механика использования заклинаний

Система заклинаний реализует следующие механики:

- Потребление маны: каждое заклинание имеет стоимость
- Ограничение радиуса: заклинания имеют дальность применения
- Получение новых заклинаний: игрок получает новое заклинание

после убийства 5 врагов

3.2. Типы заклинаний

В игре реализованы два заклинания:

Удар молнии (DirectDamageSpell):

- Прямой урон: 30 единиц
- Стоимость: 6 маны
- Радиус: 3 клетки
- Требуется точного указания цели

Ледяной вихрь (AreaDamageSpell):

- Урон по площади: 25 единиц
- Стоимость: 6 маны
- Радиус: 2 клетки
- Область: 2×2 клетки вокруг игрока

- Не требует указания цели

3.3. Механика "руки"

Рука игрока имеет следующие характеристики:

- Вместимость: 2 заклинания
- Изначально содержит одно случайное заклинание
- Автоматически дополняется при победе над пятью врагами
- Отображение доступных заклинаний через команду "3"

4. Принципы ООП, примененные в системе магии

4.1. Полиморфизм

Использование интерфейса `ISpell` позволяет обрабатывать различные типы заклинаний единообразно:

```
bool Hand::useSpell(int index, const Point& casterPosition,
                    const Point& targetPosition){
    spells[index]->cast(casterPosition, targetPosition);
    return true;
}
```

4.2. Инкапсуляция

Каждое заклинание инкапсулирует свою логику применения:

- `DirectDamageSpell` скрывает логику поиска цели и нанесения урона
- `AreaDamageSpell` скрывает логику определения попадания в область

4.3. Принцип открытости/закрытости

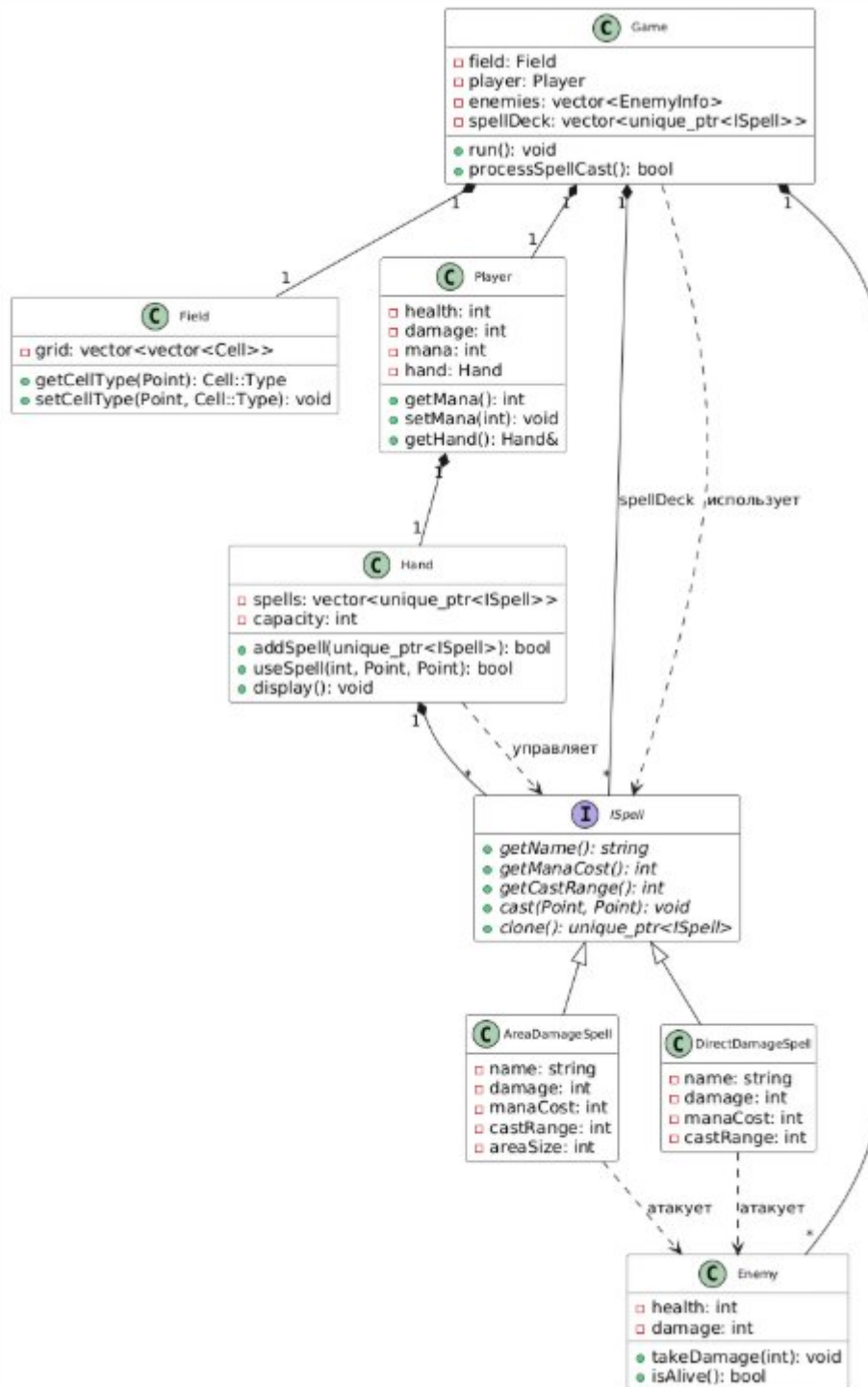
Систему легко расширить новыми заклинаниями без изменения существующего кода:

```
class NewSpellType : public ISpell {
};
```

4.4. Принцип подстановки Лисков

Все наследники `ISpell` могут использоваться везде, где ожидается базовый класс, без нарушения логики программы.

5. UML-диаграмма классов



ЗАКЛЮЧЕНИЕ

В ходе второй лабораторной работы была расширена архитектура игры добавлением системы магии.

Были реализованы:

1. Иерархия классов заклинаний
2. Система "руки" игрока для управления заклинаниями
3. Два типа заклинаний: прямой урон и урон по площади
4. Механика получения новых заклинаний

Проект демонстрирует применение принципов ООП:

- Полиморфизм через интерфейс ISpell
- Инкапсуляцию в каждом классе заклинания
- Композицию через добавление Hand в класс Player
- Принцип открытости/закрытости в расширяемой системе

заклинаний

Архитектура системы магии является готовой к дальнейшему расширению, позволяя легко добавлять новые заклинания.